STUDENT PERFORMANCE TREND ANALYSIS

1. Problem Statement

Develop a Python program to analyze a student's performance in two examinations. The program should compare marks obtained in different subjects, calculate improvements or declines in marks, find the average marks of both examinations, and identify weak subjects. The program uses Python lists, loops, arithmetic operations, and conditional statements to analyze student performance.

2. Objectives

- To store subject names and examination marks using Python lists.
- To compare marks obtained in two examinations.
- To calculate improvement or decline in each subject.
- To calculate the average marks of both examinations.
- To identify weak subjects based on examination marks.
- To understand the use of loops and conditional statements in Python.

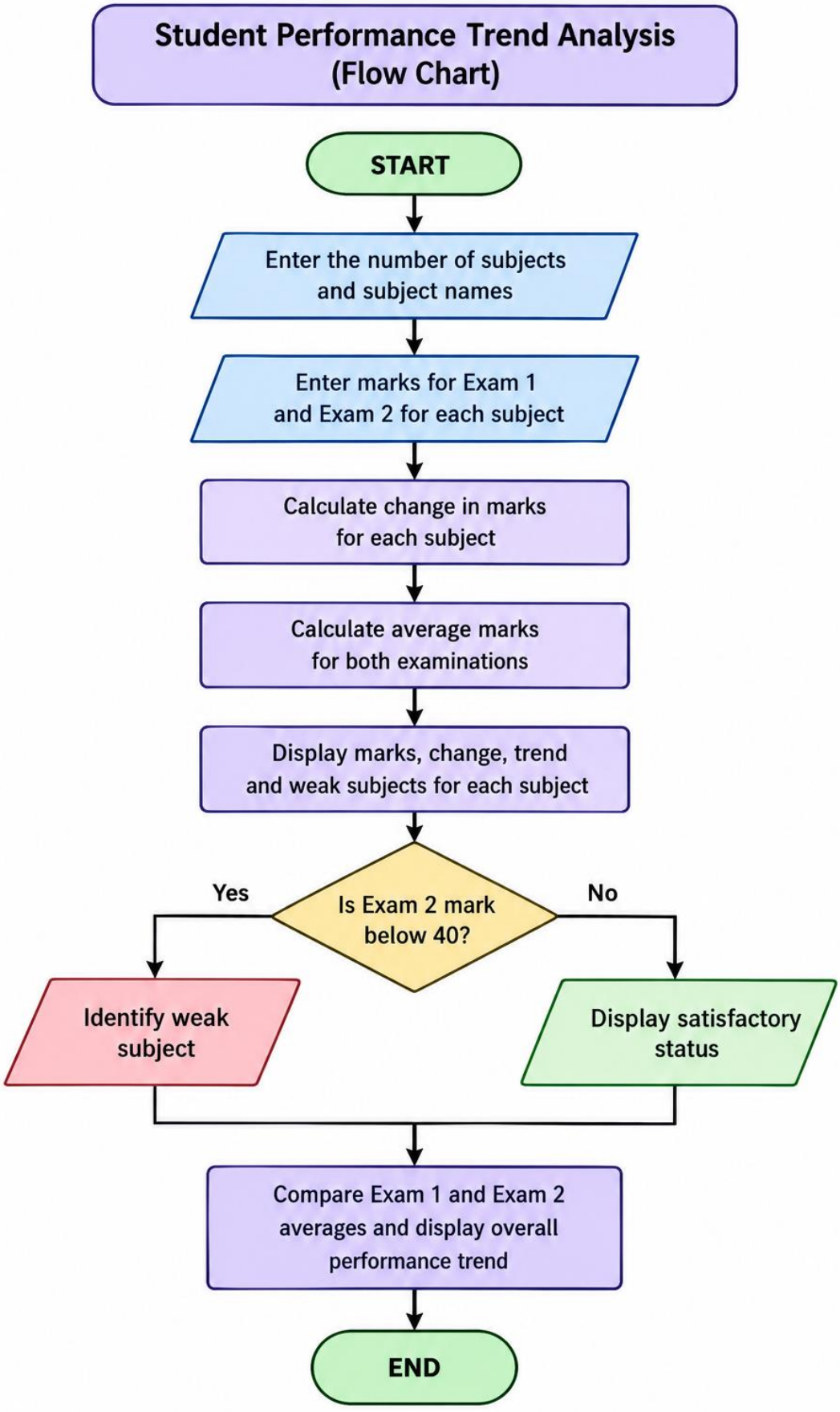
3. Assumptions

1. The student studies at least one subject.
2. Marks are collected for two examinations.
3. Marks must be between 0 and 100.
4. A subject is considered weak if the student scores below 40 in the second examination.
5. Improvement or decline is calculated by subtracting the first examination marks from the second examination marks.
6. All subject names and marks are entered by the user.

4. Algorithm

1. Start the program.
2. Ask the user to enter the number of subjects.
3. Create three lists to store subject names, Exam 1 marks, and Exam 2 marks.
4. Enter the name of each subject.
5. Input marks for both examinations.
6. Check whether the marks are between 0 and 100.
7. Calculate the change in marks for every subject.
8. Calculate the average marks of both examinations.
9. Display the marks and performance trend for each subject.
10. Identify weak subjects by checking whether Exam 2 marks are below 40.
11. Compare the two examination averages.
12. Display the overall performance trend.
13. Stop the program.

5. Flowchart


Student Performance Trend Analysis (Flow Chart)
START
Enter the number of subjects and subject names
Enter marks for Exam 1 and Exam 2 for each subject
Calculate change in marks for each subject
Calculate average marks for both examinations
Display marks, change, trend and weak subjects for each subject
Is Exam 2 mark below 40?
Yes
No
Identify weak subject
Display satisfactory status
Compare Exam 1 and Exam 2 averages and display overall performance trend
END

6. Pseudocode

START

INPUT number_of_subjects

CREATE empty lists:

    subjects

    exam1_marks

    exam2_marks

FOR each subject:

    INPUT subject_name

    INPUT exam1_mark

    INPUT exam2_mark

    STORE subject_name in subjects

    STORE exam1_mark in exam1_marks

    STORE exam2_mark in exam2_marks

SET total1 = 0

SET total2 = 0

```
FOR each subject index:
    change = exam2_marks[index] - exam1_marks[index]

    total1 = total1 + exam1_marks[index]
    total2 = total2 + exam2_marks[index]

    DISPLAY subject name and marks
    DISPLAY change in marks

    IF change > 0:
        DISPLAY "Improvement"
    ELSE IF change < 0:
        DISPLAY "Decline"
    ELSE:
        DISPLAY "No change"

    IF exam2_marks[index] < 40:
        DISPLAY "Weak subject"

average1 = total1 / number_of_subjects
average2 = total2 / number_of_subjects
```

DISPLAY average1 and average2

IF average2 > average1:

    DISPLAY "Overall performance improved"

ELSE IF average2 < average1:

    DISPLAY "Overall performance declined"

ELSE:

    DISPLAY "Overall performance unchanged"

STOP

7. **Python Actual Code**

```
def get_valid_mark(exam_name, subject):
    while True:
        try:
            mark = float(
                input(f"Enter {exam_name} marks for {subject} (0-100):
")
            )
            if 0 <= mark <= 100:
                return mark
            print("Marks must be between 0 and 100.")
        except ValueError:
            print("Please enter a valid number.")
```

```
print("STUDENT PERFORMANCE TREND ANALYSIS")

while True:
    try:
        n = int(input("Enter the number of subjects: "))
        if n > 0:
            break
        print("Enter at least one subject.")
    except ValueError:
        print("Please enter a whole number.")

subjects = []
exam1_marks = []
exam2_marks = []

for i in range(n):
    subject = input(f"Enter the name of subject {i + 1}: ")

    mark1 = get_valid_mark("Exam 1", subject)
    mark2 = get_valid_mark("Exam 2", subject)

    subjects.append(subject)
    exam1_marks.append(mark1)
    exam2_marks.append(mark2)

total1 = 0
total2 = 0

print("\n--- SUBJECT-WISE ANALYSIS ---")
```

```
for i in range(n):
    change = exam2_marks[i] - exam1_marks[i]

    total1 += exam1_marks[i]
    total2 += exam2_marks[i]

    print(f"\nSubject: {subjects[i]}")
    print(f"Exam 1: {exam1_marks[i]:.1f}")
    print(f"Exam 2: {exam2_marks[i]:.1f}")
    print(f"Change: {change:+.1f} marks")

    if change > 0:
        print("Trend: Improvement")
    elif change < 0:
        print("Trend: Decline")
    else:
        print("Trend: No change")

    if exam2_marks[i] < 40:
        print("Status: Weak subject")
    else:
        print("Status: Satisfactory")

average1 = total1 / n
average2 = total2 / n

print("\n--- OVERALL SUMMARY ---")
print(f"Exam 1 average: {average1:.2f}")
print(f"Exam 2 average: {average2:.2f}")
```

```
if average2 > average1:
    print("Overall performance improved.")
elif average2 < average1:
    print("Overall performance declined.")
else:
    print("Overall performance remained the same.")
```

8. Explanation of the Code

1. get_valid_mark() function

This function accepts marks from the user and checks that they are between 0 and 100. If the input is invalid, the program asks again.

2. Lists

The lists subjects, exam1_marks, and exam2_marks store subject names and marks for the two examinations.

3. Loops

The for loop collects marks for every subject and processes them one by one.

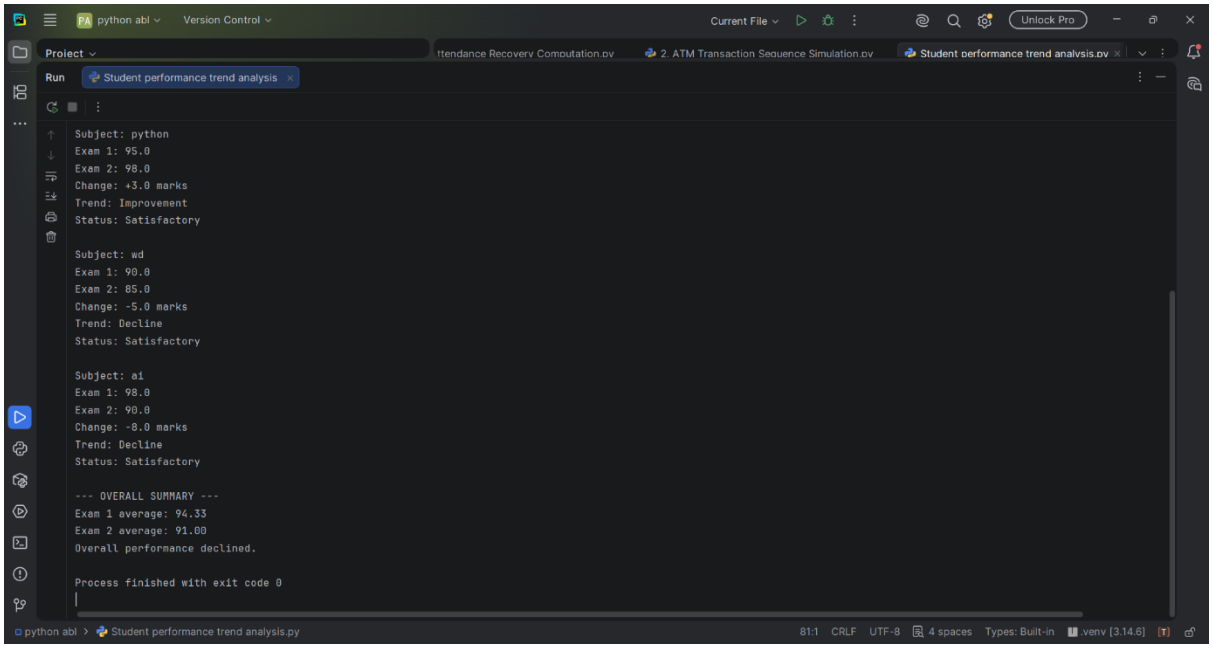
4. Change and average calculations

change = exam2_marks[i] - exam1_marks[i] calculates the difference in marks. The program adds the marks and divides each total by the number of subjects to find the averages.

5. Conditional statements

The if, elif, and else statements identify improvement, decline, no change, and weak subjects.

9. Output Screenshot



10. Conclusion

The Student Performance Trend Analysis program compares a student's marks in two examinations and calculates subject-wise improvements or declines. It also calculates average marks and identifies weak subjects. This program demonstrates the use of Python lists, loops, arithmetic operations, and conditional statements. It helps students understand their academic progress and identify subjects that require additional practice